

Unit II: Basics of Software Testing

Q1. Explain the concept of Verification and Validation

Verification and Validation are two key components of software testing that ensure software quality and correctness.

Verification – "Are we building the product right?"

- It is a static process of checking documents, design, code, and programs.
- Conducted without executing the code.
- Involves activities like **reviews, inspections, and walkthroughs**.
- Purpose is to ensure that the software meets the specified requirements and design specifications.

Example: Reviewing the SRS document to ensure all customer needs are properly captured.

Validation – "Are we building the right product?"

- It is a dynamic process where the actual software is tested.
- Performed by executing the code to ensure the final product meets user expectations.
- Involves activities like **functional testing, system testing, and user acceptance testing (UAT)**.

Example: Running the app and checking if it performs the login functionality correctly as expected by the user.

Aspect	Verification	Validation
Type	Static testing	Dynamic testing
Objective	Check if product is built correctly	Check if correct product is built
Performed by	Developers, QA engineers	Testers, End-users
Techniques used	Reviews, Inspections, Walkthroughs	Testing, Execution, UAT
When	Early in development lifecycle	After coding is complete

Q2. Explain Levels of Testing

Software testing is performed in **multiple levels**, each designed to test specific aspects of the software system.

1. Unit Testing

- Focuses on individual units or components of the software.
- Typically performed by developers.
- Ensures that each function or method performs as expected.
- Tools: JUnit (Java), NUnit (.NET), pytest (Python)

Example: Testing a login function separately with correct and incorrect credentials.

2. Integration Testing

- Tests the interaction between integrated units/modules.
- Ensures data flows correctly between components.
- Types: Big Bang, Top-Down, Bottom-Up, Sandwich

Example: Testing integration between the login module and the dashboard after successful login.

3. System Testing

- Tests the complete and integrated system.
- Verifies that the system meets all functional and non-functional requirements.
- Performed by independent QA teams.

Example: Testing the entire banking application for functionality like fund transfer, balance inquiry, etc.

4. Acceptance Testing

- Final level of testing before the product is released to the customer.
- Determines whether the system satisfies the business requirements.
- Types: Alpha Testing (by internal staff), Beta Testing (by actual users)

Example: Client tests e-commerce website to ensure all features like cart, payment, and checkout work as expected.

Level	Focus Area	Performed By	Objective
Unit Testing	Individual units/modules	Developers	Check logic correctness
Integration Testing	Interaction between units/modules	Developers/Testers	Ensure module interaction
System Testing	Entire application/system	Testers/QA	Validate system functionality
Acceptance Testing	System from end-user perspective	Customers	Ensure business requirement

Q3. Explain Testing Life Cycle in Detail [pending, here mam's answer expected]

The **Software Testing Life Cycle (STLC)** is a sequence of specific steps followed during the testing process to ensure software quality. Each phase has specific goals and deliverables.

1. Requirement Analysis

- Understand and analyze the testing requirements from the software requirement specifications (SRS).
- Identify testable and non-testable requirements.
- Stakeholders involved: Business Analyst, Test Manager, QA Team

Activities: Review requirements, Identify test environment details, Determine types of tests to be performed

Deliverables: Requirement Traceability Matrix (RTM), Automation feasibility report (if applicable)

2. Test Planning

- Defines the **strategy and scope** of testing.
- Estimates time, effort, resources, and tools required.
- Risks and mitigation plans are also identified.

Activities: Prepare the Test Plan document, Identify roles and responsibilities, Choose test tools

Deliverables: Test Plan, Effort Estimation and Schedule

3. Test Case Development

- Designing test cases and test scripts based on requirements.
- Test data is prepared during this phase.

Activities: Write test cases, Review and update them, Prepare test data

Deliverables: Test Cases, Test Scripts, Test Data

4. Test Environment Setup

- Involves preparing the hardware and software conditions needed to execute test cases.
- May run parallel to test case development.

Activities: Configure the test environment, Install test tools and applications, Validate the environment readiness

Deliverables: Environment readiness report

5. Test Execution

- Actual execution of test cases in the configured environment.
- Results are compared with expected outcomes.
- Defects are logged for failures.

Activities: Execute test cases, Log defects/bugs, Retest and regression testing

Deliverables: Test Execution Report, Defect Logs

6. Test Cycle Closure

- Marks the end of testing phase.
- Analysis of test metrics and test coverage is done.
- Final test report is prepared and shared with stakeholders.

Activities: Evaluate exit criteria, Document lessons learned, Finalize test report

Deliverables: Test Summary Report, Test Closure Report, Lessons Learned

Requirement Analysis → Test Planning → Test Case Development →
Test Environment Setup → Test Execution → Test Closure

Q4. Write Note on:

(a) Test-Driven Development (TDD)

TDD is a software development methodology where tests are written **before** writing the actual code. It follows a short development cycle: **Red → Green → Refactor**.

Phases of TDD:

1. **Write a Test:** Write a unit test for a new feature or functionality. Initially, the test will fail (Red phase).
2. **Write Minimum Code:** Write the minimal code required to pass the test. Make the test pass (Green phase).
3. **Refactor:** Clean up the code while ensuring the test still passes.

Benefits: Ensures better code quality, Reduces debugging time, Promotes modular and flexible design, Provides a full suite of automated tests

Example: Before implementing a login() function, write a test case like: `assertEquals(true, login("admin", "password"))`; Then write the actual login() code.

(b) Behavior-Driven Development (BDD)

BDD extends TDD by emphasizing **behavior** of the application from the user's perspective. It uses natural language constructs to describe test scenarios.

Key Characteristics:

- Uses "Given–When–Then" format for writing tests.
- Encourages collaboration between developers, testers, and business analysts.
- Tools: Cucumber, SpecFlow, JBehave

Example Scenario (Cucumber): Successful Login: Given User is on the login page, When User enters valid credentials, Then User should be redirected to the dashboard

Benefits: Improves communication between technical and non-technical stakeholders. Makes test scenarios easy to understand. Enhances documentation

(c) Acceptance Test-Driven Development (ATDD)

ATDD is a collaborative approach where **customers, developers, and testers** write acceptance tests **before** development begins.

Key Features:

- Focus on capturing **user acceptance criteria**
- Tests are written from the end-user perspective
- Similar to BDD but more business-focused

Example: Before developing the "add to cart" feature: Test: When the user clicks on 'Add to Cart', the item should appear in the cart.

Benefits: Helps ensure the software meets business requirements. Enhances team collaboration. Encourages shared understanding of features

Approach	Focus	Who Writes the Tests	Format/Tool	Primary Benefit
TDD	Code correctness	Developers	Unit tests	Early bug detection
BDD	System behavior	Developers + Testers + BAs	Gherkin/Cucumber	Better communication
ATDD	User acceptance	Customer + Dev + QA	Acceptance tests	Validates business needs

Q5. Develop Test Cases for Online Shopping System [table format should be same as mam's format]

Objective: To ensure the online shopping system works correctly by validating key functionalities through test cases.

Module: User Login

Test Case ID	Description	Input	Expected Output	Status
TC_Login_01	Valid login credentials	Username: user01, Password: abc123	Redirect to homepage	Pass/Fail
TC_Login_02	Invalid password	Username: user01, Password: wrong	Show error message	Pass/Fail
TC_Login_03	Empty fields	Username: "", Password: ""	Prompt to fill fields	Pass/Fail

Module: Product Search

Test Case ID	Description	Input	Expected Output	Status
TC_Search_01	Search existing product	"Laptop"	Display relevant products	Pass/Fail
TC_Search_02	Search non-existing product	"Flying car"	Show "No results found"	Pass/Fail

Module: Add to Cart

Test Case ID	Description	Input	Expected Output	Status
TC_Cart_01	Add product to cart	Product ID: 101	Product added to cart	Pass/Fail
TC_Cart_02	Add out-of-stock item	Product ID: 205	Show "Out of stock" message	Pass/Fail

Module: Checkout & Payment

Test Case ID	Description	Input	Expected Output	Status
TC_Pay_01	Valid card payment	Card details, Address	Order confirmation	Pass/Fail
TC_Pay_02	Invalid card number	Card number: 1234	Show payment error	Pass/Fail
TC_Pay_03	No delivery address	Card info without address	Prompt to enter address	Pass/Fail

Module: Order History

Test Case ID	Description	Input	Expected Output	Status
TC_Hist_01	View previous orders	User ID: user01	List of past orders	Pass/Fail
TC_Hist_02	No order history	New user	"No orders found" message	Pass/Fail

Additional Suggestions: Include edge cases (e.g., SQL injection in input fields). Add test cases for **session timeout**, **password recovery**, and **multi-language support**, if applicable.

Q6. Construct Test Plan for Online Shopping System [test plan format should be same as mam's format]

A **Test Plan** is a detailed document that outlines the strategy, scope, schedule, resources, and activities required for testing a software application. It ensures that all aspects of testing are covered systematically.

1. Test Plan Identifier: TP_OSS_01 (Online Shopping System Test Plan v1.0)

2. Introduction: This test plan outlines the approach to verify and validate the core functionalities of the Online Shopping System (OSS), including user login, product search, cart operations, payment, and order management.

3. Objectives and Tasks

- To ensure the application meets functional and non-functional requirements.
- To identify and fix defects before product release.

4. Scope

In Scope:

- User registration/login
- Product browsing and search
- Cart and checkout
- Payment gateway integration
- Order history and tracking

Out of Scope:

- Backend database optimization
- Third-party inventory systems

5. Test Items

- OSS Web Interface
- Product Listing
- Shopping Cart
- Payment Module
- User Profile and Order Management

6. Features to be Tested

- Secure Login/Logout
- Functional Search Bar
- Accurate Add-to-Cart operations
- Payment Processing with valid and invalid details
- Order placement and history view

7. Features Not to be Tested

- Email delivery timing for order confirmation
- Performance under heavy load (unless specified)

8. Test Strategy

- **Manual Testing** for UI and core functionality
- **Automation** for regression testing using Selenium
- **Unit Testing** by developers using JUnit
- **System & Integration Testing** by QA team

9. Test Deliverables

- Test Plan Document
- Test Cases and Scenarios
- Test Summary Report
- Defect Report
- Final Test Closure Report

10. Testing Schedule

Phase	Start Date	End Date
Test Planning	01-Aug	03-Aug
Test Case Design	04-Aug	07-Aug
Test Execution	08-Aug	15-Aug
Defect Reporting	08-Aug	16-Aug
Test Closure	17-Aug	18-Aug

11. Entry and Exit Criteria

Entry Criteria:

- Code freeze completed
- Test environment ready
- Test cases reviewed

Exit Criteria:

- All critical test cases executed
- No critical or high-severity bugs remain
- Test summary and closure report submitted

12. Resources

- 2 Test Engineers
- 1 Automation Tester
- 1 Test Lead

13. Risks and Contingencies

- **Risk:** Third-party API may change during testing.
- **Mitigation:** Maintain mock APIs for testing.

Q7. Explain Bug Life Cycle [diagram should be draw as mam given]

The **Bug Life Cycle** (also called **Defect Life Cycle**) refers to the sequence of stages that a bug or defect goes through from its identification to its closure in the software development and testing process. Understanding the bug life cycle is essential to effectively track and manage defects and ensure timely resolution.

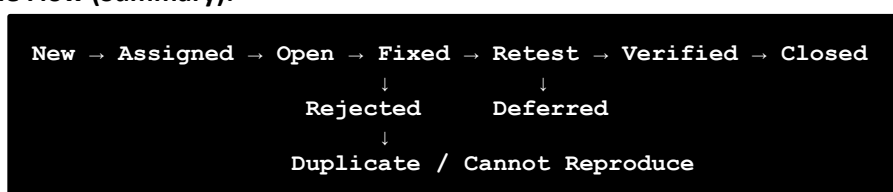
Stages of Bug Life Cycle:

1. **New:** When a tester finds a defect and logs it in the defect tracking tool (e.g., JIRA, Bugzilla), it is assigned the status *New*. Example: Tester finds that the payment button does not work after adding a product to cart.
2. **Assigned:** The bug is assigned to a developer or team lead for analysis and fixing.
3. **Open:** The developer begins working on the bug. The status is set to *Open*. At this stage, the developer investigates the root cause.
4. **Fixed:** After the developer fixes the issue, the status is changed to *Fixed*. The defect is now ready for testing.
5. **Retest:** The tester retests the application to verify if the bug is actually fixed.
6. **Verified:** If the tester confirms the bug no longer exists, the status is set to *Verified*.
7. **Closed:** Once verified, the bug is marked as *Closed*. This means no further action is required.

Other Possible States:

- **Rejected:** Developer decides the bug is invalid or not reproducible.
- **Duplicate:** Bug is already reported; hence marked as duplicate.
- **Deferred:** Bug fixing is postponed due to low priority or future release.
- **Cannot Reproduce:** Tester or developer cannot replicate the bug in the described steps.

Bug Life Cycle Flow (Summary):



Importance of Bug Life Cycle:

- Helps in tracking status of defects.

- Ensures proper communication between QA and development teams.
- Supports decision-making in release planning.
- Prevents unresolved or lost bugs.

Q8. What is Defect Repository? Explain with Suitable Example

A **Defect Repository** is a **centralized database or storage system** where all the information related to identified software defects (bugs) is collected, stored, tracked, and managed. It allows the software testing and development teams to keep a systematic record of all defects throughout the software development lifecycle.

It serves as a **reference base** for: Future testing cycles, Process improvement, Risk assessment, Reporting and analysis

Contents of a Defect Repository: Each entry in a defect repository typically contains:

Field	Description
Defect ID	Unique identifier for the defect
Title/Summary	Short description of the defect
Description	Detailed explanation of the issue
Steps to Reproduce	Exact steps that lead to the defect
Environment	Platform, browser, OS where defect was found
Severity & Priority	Impact level and urgency to fix
Status	Current state (e.g., New, Fixed, Closed)
Assigned To	Developer or team responsible for resolving the defect
Date Created & Modified	Timestamp to track progress
Attachments/Screenshots	Visual proof or logs to assist developers

Tools to Maintain a Defect Repository: JIRA, Bugzilla, Redmine, MantisBT, HP ALM (Application Lifecycle Management)

Example: Let's say a tester finds that a **"Checkout" button on an e-commerce website is not working**. The defect repository might contain the following entry:

- **Defect ID:** BUG2025-001
- **Title:** Checkout button not responding
- **Description:** When user clicks on the checkout button, nothing happens. No errors are shown.
- **Steps to Reproduce:**
 1. Add any product to cart
 2. Go to cart page
 3. Click on "Checkout"
- **Environment:** Chrome v116, Windows 11
- **Severity:** High
- **Priority:** Critical
- **Status:** New
- **Assigned To:** Developer_A
- **Date Created:** 5th August 2025

Benefits of a Defect Repository:

1. **Centralized Access:** All team members can view and update defect statuses.
 2. **Traceability:** Helps in tracking defect origin and resolution.
 3. **Accountability:** Assigns ownership and deadlines.
 4. **Historical Data:** Supports future project estimations and test coverage analysis.
 5. **Quality Metrics:** Helps generate reports on defect density, open vs. closed bugs, etc.
-

Q9: Explain Testing Life Cycle in detail [see Q3]

Q10. Develop Test Case for Username and Password

Testing **username and password fields** is a critical part of login or authentication systems. Below is a well-structured test case table that considers **both positive and negative scenarios**.

Test Case Table:

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Actual Result	Status
TC_01	Valid login credentials	Enter valid username and password, click login	Username: ajit123 Password: Ajit@2025	User should be redirected to dashboard/homepage	As expected	Pass
TC_02	Empty username	Leave username field blank, enter valid password, click login	Username: Password: Ajit@2025	Error message: "Username is required"	As expected	Pass
TC_03	Empty password	Enter valid username, leave password field blank, click login	Username: ajit123 Password:	Error message: "Password is required"	As expected	Pass
TC_04	Both fields empty	Leave both fields blank and click login	Username: Password:	Error: "Username and Password are required"	As expected	Pass
TC_05	Invalid username	Enter incorrect username and valid password	Username: wrongUser Password: Ajit@2025	Error: "Invalid username or password"	As expected	Pass
TC_06	Invalid password	Enter valid username and incorrect password	Username: ajit123 Password: Wrong@123	Error: "Invalid username or password"	As expected	Pass
TC_07	SQL Injection attempt	Enter admin' -- or OR '1'='1' as username or password	Username: admin' -- Password: anything	Error: "Invalid input" / prevent login	As expected	Pass
TC_08	Password field masked	Check if password field displays dots instead of characters	Type: Ajit@2025	Password characters should be hidden	As expected	Pass
TC_09	Max character length check	Enter 256 characters in username and password fields	Long strings	Error or truncate at limit (as per spec)	As expected	Pass
TC_10	Special characters in username/password	Enter special characters in both fields	Username: ajit!@# Password: Ajit@#123	Accept if allowed or show validation error	As per rules	Pass/ Fail

Notes:

- Validation should occur on both **client-side and server-side**.
- Passwords should follow complexity rules like **uppercase, lowercase, number, and special character**.
- Error messages should be **user-friendly** and **non-revealing** (e.g., do not say "username correct but password wrong").